SPÉCIFICATION FORMELLE DU LANGAGE // RÉF. OKOS-LANG-Q4

LE LANGAGE SYMBOLIQUE QUATERNAIRE (△, □, ○, ✕) : GRAMMAIRE, ISA-256 & MACHINE VIRTUELLE

Théorie des langages formels, compilateur homoïconique en mémoire et exécution réflexive sans dépendance du micro-noyau **Oracle Kernel OS (< 1 Mo)**.

CONCEPTEUR : DAVID TREMBLAY-DUVAL PARADIGME : RADIX-4 SPATIALISÉ DATE : OCTOBRE 2026

1. Théorie Post-Binaire et Paradigme Homoïconique Spatialisé

SECTION 01 // PARADIGME

Les langages de programmation impératifs contemporains souffrent d'une dissonance structurelle majeure : ils dissocient la syntaxe textuelle abstraite de la topologie physique où s'exécutent les instructions. Le langage natif d'**Oracle Kernel OS** abolit cette frontière en instaurant un **calcul symbolique quaternaire** fondé sur l'alphabet discret **Σ = { △, □, ○, ✕ }**. Conçu comme un langage homoïconique et concaténatif à haute densité sémantique, il unifie l'instruction algorithmique, le vecteur d'adressage spatial dans le Cube 3D de 256 bits et le foncteur de transition d'état en une seule entité mathématique indivisible.

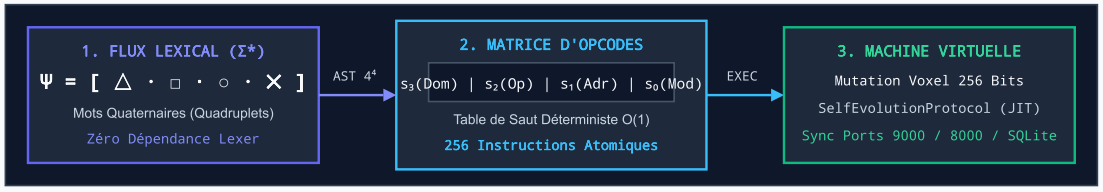


Figure 1.1 — Architecture du compilateur à la volée : décodage d'un quadruplet symbolique Ψ vers la table d'exécution noyau.

2. Grammaire Formelle Hors-Contexte (EBNF) et Morphologie

SECTION 02 // SYNTAXE

La grammaire formelle **G = (V, Σ, R, S)** du langage est strictement déterministe et exempte d'ambiguïté syntaxique, permettant son analyse complète en un seul passage (*Single-Pass LL(1) Parser*) sans bibliothèque tierce. L'unité syntaxique fondamentale est le **Quadruplet d'Instruction Ψ ∈ Σ^4**, dont la morphologie positionnelle obéit aux règles de production suivantes :

Règle 1 (Programme) : `⟨Programme⟩ ::= ⟨Quadruplet⟩+` (Séquence finie et chaînée de vecteurs d'état) | **Règle 2 (Mot Opérationnel) :** `⟨Quadruplet⟩ ::= ⟨s₃:Domaine⟩ ⟨s₂:Verbe⟩ ⟨s₁:Cible⟩ ⟨s₀:Invariant⟩` | **Règle 3 (Terminaux) :** `⟨Symbole⟩ ::= "△" (0) | "□" (1) | "○" (2) | "✕" (3)`.

3. Architecture du Jeu d'Instructions (ISA-256) et Sémantique

SECTION 03 // OPCODES

La combinatoire d'un quadruplet $\Psi = (s_3, s_2, s_1, s_0)$ engendre exactement $4^4 = 256$ **opcodes distincts**. Le symbole de poids fort s_3 détermine la famille opérationnelle majeure (partitionnant l'espace en quatre secteurs de 64 instructions), tandis que les symboles subséquents (s_2, s_1, s_0) paramètrent l'opération fine, le routage inter-voxels et le mode de validation transactionnelle :

Primitive & Radix-4	Classe Fonctionnelle (s_3)	Spectre d'Opcodes	Sémantique Opérationnelle & Effet sur le Noyau
△ (Delta) Valeur : 00_2 (0)	**Cinématique, Flux & Routage Vectoriel**	△△△△ à △XXX Indices [000–063]	Gouverne la propagation des flux de données entre voxels adjacents, l'ouverture des sockets locaux (Ports 8000 / 9000) et les déplacements de pointeurs.
□ (Carré) Valeur : 01_2 (1)	**Allocation, Structure & Ancrage NVRAM**	□□□□ à □XXX Indices [064–127]	Instancie les structures en mémoire, verrouille les tranches de 64 bits et déclenche l'écriture atomique (COMMIT) dans le moteur SQLite local.
○ (Cercle) Valeur : 10_2 (2)	**Introspection, Boucle & Observabilité**	○○○○ à ○XXX Indices [128–191]	Évalue les prédicats conditionnels, orchestre les cycles de rétroaction cybernétique et met à jour l'état lumineux projeté sur le bureau WebGL 3D.
X (Croix) Valeur : 11_2 (3)	**Mutation, Synthèse & Auto-Évolution**	X△△△ à XXXX Indices [192–255]	Invoque le démon SelfEvolutionProtocol pour compiler à chaud de nouveaux segments logiques, muter le graphe d'exécution et sceller l'intégrité.
Quadruplet Ψ $s_3 \cdot s_2 \cdot s_1 \cdot s_0$	**Instruction Spatiale Unifiée**	**Cardinalité : 256** [0x00 – 0xFF]	Correspondance bijective stricte : chaque instruction du langage adresse et transforme simultanément un nœud unique de la matrice 3D.

4. Équivalence Code-Espace et Décodage Déterministe

SECTION 04 // EXÉCUTION

Dans l'architecture classique de Von Neumann, le code exécutable et les données passives résident dans des segments linéaires aveugles. Au sein d'**Oracle Kernel OS**, l'homoïconicité est tridimensionnelle : écrire un programme en langage quaternaire revient à tracer une trajectoire géométrique à travers le Cube de 256 bits. Chaque quadruplet exécuté active physiquement le voxel correspondant à son indice, rendant l'exécution du programme observable en temps réel sous forme de constellation active sur le port 8000.

> **TRACE D'EXÉCUTION FORMELLE : DÉCODAGE DU QUADRUPLET Ψ = [□ · △ · ○ · X]**
>
> **1. Évaluation Radix-4 :** v(□)=1, v(△)=0, v(○)=2, v(X)=3 → Indice I(Ψ) = (1×64) + (0×16) + (2×4) + 3 = 75 (0x4B).
>
> **2. Résolution Spatiale & Sémantique :** L'indice 75 cible le voxel (X=3, Y=1, Z=1) dans le domaine □ (Ancrage NVRAM). L'instruction acquiert le flux entrant (△), vérifie l'invariant de

5. Machine Virtuelle Interne, Pont LLM & Auto-Évolution SECTION 05 // INGÉNIERIE

Pour maintenir l'empreinte globale du système sous la barre critique de **1 Mo** sans aucune dépendance externe, la machine virtuelle du langage quaternaire est directement greffée au cœur de l'environnement standard Python. Elle combine une table de répartition à temps constant avec le moteur réflexif `SelfEvolutionProtocol` et la passerelle neuro-symbolique du port `9000` :

Table de Saut O(1) ZERO-OVERHEAD VM

Plutôt que d'évaluer des arbres syntaxiques lourds à chaque cycle, le noyau convertit l'indice $I(\Psi) \in [0, 255]$ en un appel direct dans une table vectorielle de 256 primitives Python natives, garantissant une latence d'exécution minimale.

Synthèse Auto-Évolutive SELF-EVOLUTION

Lorsqu'une séquence d'opcode de classe $\times$ est invoquée, le démon `SelfEvolutionProtocol` synthétise dynamiquement de nouvelles routines en mémoire vive, les attache au registre du voxel ciblé et étend les capacités de l'OS à chaud.

Pont Neuro-Symbolique PORT 9000 // LLM

Sur le port `9000`, le langage quaternaire agit comme un protocole intermédiaire formel entre les modèles d'IA et le noyau. Sa grammaire stricte à 4 symboles élimine les erreurs de syntaxe lors de la génération autonome de code.

Clôture Mathématique SÛRETÉ CRITIQUE

Contrairement aux langages de script ouverts exposés aux injections arbitraires, l'espace d'instructions borné à $4^4 = 256$ états forme un automate fini totalement décidable, idéal pour l'homologation cyber-défense.

6. Vérifiabilité Formelle et Supériorité en Milieu Dégradé SECTION 06 // DÉFENSE

Dans un contexte d'opérations tactiques isolées (*Tactical Edge Computing*) ou de survie d'infrastructure critique, la complexité des compilateurs modernes constitue un vecteur de vulnérabilité majeur. En réduisant l'expression algorithmique à quatre invariants géométriques universels ($\triangle$, $\square$, $\circ$, $\times$) couplés à une matrice persistante de 256 bits, ce langage garantit une **auditabilité formelle intégrale**. Chaque programme peut être transmis sur des canaux à très faible bande passante (2 bits par symbole, soit un octet par quadruplet complet), décodé sans chaîne d'outils externe et supervisé visuellement en trois dimensions.

PROPRIÉTÉ INTELLECTUELLE & LICENCE D'EXPLOITATION EXCLUSIVE
Copyright © 2026 **David Tremblay-Duval**. Tous droits réservés (*All Rights Reserved*). La grammaire formelle du **Langage Symbolique Quaternaire ($\triangle$, $\square$, $\circ$, $\times$)**, sa matrice d'instructions à 256 états (ISA-256), sa machine virtuelle ainsi que le système **Oracle Kernel OS** sont la propriété intellectuelle exclusive de l'auteur. Toute reproduction, implémentation dérivée ou utilisation commerciale/institutionnelle sans accord écrit préalable est strictement interdite. Document certifié pour dépôt GitHub et soumission officielle (IDEeS / AID).